

Density-Based Traffic Signal Control

Change Log: Broker Migration, Emergency Preemption, and RTX 2060 Performance Tuning

Engineering note

2026-06-15

1 Summary

This document records the changes made to the density-based traffic signal control system in the `tl` repository. The work covers four areas:

1. Migrating the MQTT broker from `test.mosquitto.org` to `broker.hivemq.com` across the firmware and documentation.
2. Confirming and documenting the two-lane, density-driven signal logic (one light for the vertical lane, one for the horizontal lane).
3. Adding *emergency-vehicle preemption*: an ambulance or firetruck forces its lane green, but only once we are confident the detection is real.
4. Performance optimizations targeting a client GPU no larger than an NVIDIA RTX 2060 running the smallest YOLO model, while keeping both the live-camera (RTSP) and video-file pipelines intact.

2 System Context

The pipeline captures video from four IP cameras (or local video files for testing), runs a custom-trained YOLO11 detector (classes `ambulance`, `firetruck`, `vehicle`), counts vehicles per direction, and decides which of the two traffic lights should be green. Decisions are published over MQTT to topics `ub-traffic-light/signals/vertical` and `.../horizontal`. An ESP32 subscribed to each topic drives the physical lights and coordinates the partner light over ESP-NOW.

3 1. MQTT Broker Migration to HiveMQ

The Python publisher (`gui/publisher.py`) and the manual test scripts already pointed at `broker.hivemq.com`. The remaining references to `test.mosquitto.org` were updated so the whole system is consistent:

- `esp/bidirectional_mqtt/bidirectional_mqtt.ino`
- `esp/bidirectional_mqtt_vertical/bidirectional_mqtt_vertical.ino`
- `esp/bidirectional_mqtt_horizontal/bidirectional_mqtt_horizontal.ino`
- `install.txt` (the `MQTT_SERVER` configuration line)

Each `const char* MQTT_SERVER` now reads `"broker.hivemq.com"` on port 1883. No topic names or client-ID logic changed, so the existing firmware behaviour is preserved.

4 2. Two-Lane Density Logic

The intersection has exactly two phases. Cameras are tagged with a `direction` of `vertical` or `horizontal`; collinear lanes share a phase. Per inference cycle the system sums the `vehicle` detections for each direction and prefers to give green to the busier direction:

```
if vertical_count > horizontal_count: -> green: vertical
elif horizontal_count > vertical_count: -> green: horizontal
else:                                  -> hold current phase
```

Two timing guards keep the lights sane and safe:

- **Minimum green** (`MIN_GREEN`, 15 s): the system will not switch away from a freshly granted green until this elapses, preventing flicker when counts are close.
- **Maximum green** (`MAX_GREEN`, 85 s): if one direction has held green this long, the opposite direction is forced green to avoid starvation.

Every phase change is mediated by a 3 s yellow transition on the lane losing priority, after which its partner light is told to go green.

5 3. Emergency-Vehicle Preemption

5.1 Goal

If an ambulance or firetruck is present in a lane, that lane should turn green immediately. However, a single noisy frame must *not* hijack the intersection, so a detection only counts as an emergency once we are confident it is genuine.

5.2 Confidence gate

A detection is treated as a candidate emergency only if its class is `ambulance` or `firetruck` *and* its confidence is at least `EMERGENCY_CONFIDENCE` (default 0.55, higher than the 0.30 used for ordinary vehicle counting).

5.3 Persistence gate

Per cycle, each lane's count of confident emergency detections updates a *streak* counter: the streak increments while emergencies are seen and resets to zero the moment a lane shows none. A lane is only declared a confirmed emergency once its streak reaches `EMERGENCY_FRAMES` (default 5 consecutive cycles). This rejects isolated false positives while still reacting within a fraction of a second at typical frame rates.

5.4 Preemption behaviour

When a lane is confirmed:

- It is switched green immediately, **bypassing the minimum-green hold** on the current phase.
- While its emergency remains active it is **exempt from the maximum-green timeout**, so it stays green until the emergency vehicle clears.
- The 3 s yellow transition on the losing lane is still honoured for safety.
- If both lanes show confirmed emergencies simultaneously, the currently green lane is held; otherwise the lane with more emergency detections wins.

5.5 Tuning

Both thresholds are configurable per deployment via the `experimental_settings` block in the camera config files (`emergency_confidence`, `emergency_frames`) and are applied live when settings change. The detector classes `ambulance` and `firetruck` can be enabled in the on-screen bounding-box overlay for visual verification.

6 4. Performance Optimizations (RTX 2060, Smallest Model)

The clients run on a GPU no larger than an RTX 2060 with the smallest YOLO weights, so the goal was to cut per-frame overhead without changing detection behaviour.

6.1 Eliminated redundant GPU→CPU transfers

Previously each frame’s detections were pulled off the GPU as separate `xyxy`, `conf`, and `cls` tensors in the display path, the counting path performed its own `cls` transfer, and the inference helper computed shifted coordinates that were then *discarded*. The detections are now transferred **once** per frame as a single (N,6) array `[x1,y1,x2,y2,conf,cls]`, already shifted into full-frame coordinates, and cached. Both the signal logic and the renderer read this cache. The dead transfer was removed entirely. On four cameras this removes several host/device synchronisations per cycle, the most costly non-inference work in the loop.

6.2 cuDNN autotuning, TF32, and FP16

Because the inference tile size is fixed frame-to-frame, cuDNN benchmarking is enabled so it caches the fastest convolution kernels. TF32 matmul/convolution paths are enabled, and the model is fused and run in FP16 on the GPU. These are low-risk wins well suited to Turing-class hardware.

6.3 Warm-up inference

A single dummy inference is run at startup so the first real frame no longer pays the one-off kernel-compilation and allocator stall. This also makes the reported latency reflect steady-state performance.

6.4 Correct per-camera ROI indexing

The region-of-interest cropping now uses each frame’s true camera index rather than its position within the (possibly shorter) batch. This fixes a latent bug where a dropped camera frame would apply the wrong ROI, and keeps inference confined to the relevant road area.

6.5 Shared pipeline path (*the other implementation, kept*)

The repository has two capture paths: live RTSP cameras and local video files (used for testing/demo). Both were retained. Their per-cycle processing (batching, inference, signal update, rendering) was de-duplicated into a single `_process_frames` helper, so both paths now share the identical, optimized code and cannot drift apart. The video-file path additionally benefits from all the optimizations above.

7 Files Changed

- `gui/inference.py` — emergency preemption, single-transfer detection cache, cuDNN/TF32/FP16 setup, warm-up, ROI index fix, shared `_process_frames` helper, live-tunable thresholds.
- `gui/publisher.py` — already on HiveMQ (no change required).
- `esp/**/*.ino` — broker set to `broker.hivemq.com`.
- `configs/*.json` — added `emergency_confidence` and `emergency_frames` to `experimental_settings`.
- `install.txt` — broker documentation updated.

8 Validation Notes

Python modules were syntax-checked and the configuration files were validated as JSON. End-to-end behaviour (live detection latency on the RTX 2060 and the emergency override against real ambulance/firetruck footage) should be confirmed on the target hardware, and `EMERGENCY_CONFIDENCE` / `EMERGENCY_FRAMES` tuned to the deployed camera frame rate.